

Homework 1 Report

1. Problem Statement

The objective of this assignment is to develop an AI agent capable of processing multiple images of supermarket receipts. This agent will be built upon the Gemini multimodal large language model. The system must extract data to respond to two types of queries: total spending amount and total pre-discount amount. Additionally, the system must possess the capability to reject irrelevant queries.

2. Solution

I adopted a sequential processing strategy instead of feeding all images into the context window simultaneously, which yields higher accuracy.

2.1 System Architecture

This solution is implemented using Python and the `langchain_google_genai` library.

Image Preprocessing: A helper function converts JPG images into Base64 data URLs for compatibility with the Gemini API.

Sequential Extraction: The system iterates through the receipt image list, sending specific prompts to the LLM for each image.

Aggregation Processing: Python logic aggregates (sums) numerical results extracted from each image to generate the final answer.

2.2 Prompt Engineering

Core performance relies on differentiated prompts for distinct query types. `SystemMessage` defines the agent's behavioral norms and constraints.

System Directive: “You are an intelligent assistant analyzing supermarket receipts. For questions involving money/cost, return only the final numerical value (float). If a user's query is unrelated to the receipt, respond with ‘This question is unrelated to the receipt and cannot be answered’” (This mechanism fulfills the requirement to reject irrelevant queries).

Query 1: Identify the final charge amount.

Prompt: “What is the total amount paid in this receipt? (Net total)”

Query 2 (Total Amount):

Requires calculating the pre-discount price. The model will first locate the "Total Savings Amount" and add it back to the payment amount.

Prompt: "If no discounts applied, how much would I have paid? Locate the 'Total Savings Amount' and add it to the 'Total Payment Amount'. Return only the final value."

2.3 Post-Processing

To prevent parsing errors, I implemented a Regular Expression (re.search) cleaner. Even if the LLM returned text like "The total is 50.00", the code automatically extracts 50.00 and converts it to a float for calculation.

3. Implementation and Results

This solution has been tested against the benchmark data provided in the evaluation code block.

Query 1 Results: The model successfully identified the net payment amount for all 7 receipts.

Query 2 Results: The model successfully inferred the original price (net price + discount amount).

Conclusion

By combining the visual reasoning capabilities of Gemini-2.5-flash with a structured, iterative Python workflow, the agent successfully performed the accounting task. The use of strict system instructions ensured the output was machine-readable and robust against irrelevant inputs.

Disclaimer: In this report, I utilized DeepL for translation assistance and Gemini for structural organization.